显式静电 PIC 模块通用化：实现路径与可复用架构设计

（优先：量纲体系、模块字典规范、MCC 碰撞/反应；面向隐式 PIC 与电磁扩展）

陈冲（Chong Chen）

2026-01-23

摘要

本文给出一个可执行的“从现有 PaperA 显式静电 PIC 流程（deposit → Poisson → E → gather → push → output）到通用可用模块”的实现路径。考虑到后续要扩展隐式 PIC、电磁（EM）与更多物理模块，路线以“先抽出可复用基础设施”为原则：**量纲/归一化（Units）、模块字典规范与校验（Schema/-Validator）、碰撞/反应 MCC 框架（Cross section + Reaction DSL + Operator 接口）**优先落地；随后再补齐边界/源项/诊断/算例库，最终形成可回归、可复现、可扩展的模块体系。

目录

1 总体目标与设计原则 **1**

1.1 目标（验收口径） 1

1.2 核心原则 1

2 建议的可复用架构分层（面向后续隐式/电磁扩展） **2**

2.1 分层与职责 2

2.2 关键接口（建议最小集合） 2

3 实现路径（一步一步）：优先 Units、字典规范、MCC **3**

4 字典规范建议（面向扩展：ES/Implicit/EM 共用骨架） **6**

5 推荐的最短落地顺序（浓缩版） **8**

6 结语 **8**

1 总体目标与设计原则

1.1 目标（验收口径）

面向“IEC 分析 + CPC/经典 benchmark + PaperA 算例”，模块达到：

- **输入可表达真实工况：**几何/网格由 `geometry.dict` 与 `mesh.dict` 描述；显式 PIC 模块字典可用 SI 输入（可选内部归一化）。

- **物理闭环可配置：**物种、源项、壁面、碰撞/反应、外电路（至少浮动电极/电流平衡）可通过字典组合。
- **诊断与回归完备：**能量、Gauss 残差、壁面电流、反应率、能谱等作为 “golden outputs” 可回归。
- **架构可扩展：**同一套 Units/字典/碰撞/诊断框架可复用于隐式 PIC 与 EM 模块。

1.2 核心原则

1. **先基础设施、后堆物理：**先把 Units + Dict Schema + MCC 框架做扎实，再做更多边界/源项/算例。
2. **接口先行：**Deposition/Gather、Pusher、FieldSolver、CollisionOperator、BoundaryModel、SourceModel 都以接口形式落地，显式/隐式/EM 仅替换实现。
3. **算例驱动：**每一步都有最小验收算例（benchmark）与固定随机种子，防止 “跑得动但变味”。

2 建议的可复用架构分层（面向后续隐式/电磁扩展）

2.1 分层与职责

建议将 PIC 相关复用能力沉淀为 “PIC Core”（不依赖具体显式/隐式算法），并由不同 Module 组合：

- **Core/IO：**字典解析、Schema 校验、单位解析与打印、日志。
- **Core/Units：**SI $\leftrightarrow$ CodeUnits 的转换与参考尺度管理。
- **PICCore/Particles：**物种、宏粒子权重、分布采样、RNG、重启序列化。
- **PICCore/Operators：**Deposit/Gather、Pusher、CollisionOperator、SourceOperator、BoundaryOperator。
- **FieldCore/Solvers：**Poisson（ES）、Maxwell（EM）等 “FieldSolver” 接口与实现；线性求解器统一接口。
- **Diagnostics：**全局守恒、残差、壁面通量、能谱、反应统计；统一输出到现有 OutputManager。

2.2 关键接口（建议最小集合）

```
// 统一单位/尺度
struct UnitsContext { /* SI<->code, reference scales, printing */ };

// 模块运行时上下文（可被不同模块共享）
struct SimContext {
  const Mesh& mesh;
  UnitsContext units;
```

```
  RNG rng;
  Diagnostics diag;
  // ... output, timers, etc.
};

// 场求解器（ES/EM/Implicit都能复用）
class IFieldSolver {
public:
  virtual void assemble(const SimContext&, const FieldRHS& rhs) = 0;
  virtual void solve(SimContext&, FieldState& state) = 0;
  virtual ~IFieldSolver() = default;
};

class IDepositOperator { /* particles -> grid rho */ };
class IGatherOperator  { /* grid E/phi -> particles */ };
class IPusher          { /* advance x,v */ };
class ICollisionOperator { /* MCC events */ };
class ISourceOperator  { /* inject/ionize/recombine */ };
class IBoundaryOperator{ /* wall interaction + current/SEE */ };
```

3 实现路径（一步一步）：优先 Units、字典规范、MCC

下面按 “Step 0–12” 给出推荐顺序。每一步都包含：**目的证明（Why）**、**交付物（Deliverables）**、**验收算例（Acceptance）**。

Step 0：冻结基线与回归骨架（1 次性）

Why：避免后续重构导致 “能跑但结果漂移”。
Deliverables：

- 建立 `cases/` 目录：`paperA/`, `bench/`, `iec/` 三类。
- 统一 “run script”（本地/CI 均可），固定 RNG seed。
- 现有流程输出最小诊断：总电荷、Gauss 残差、场能/动能。

Acceptance：PaperA 至少 1 个算例跑通并生成基线曲线文件（作为 golden）。

Step 1：Units/Normalization 工具层（优先）

Why：真实工况输入必须有量纲；内部可选无量纲以保持数值健康。
Deliverables：

1. **UnitsContext**：支持 SI 与 CodeUnits 双模式；记录 $L_0, T_0, \Phi_0, n_0, v_0, E_0, \rho_0$ 等参考量。
2. 统一 “单位解析”：字典字段支持 “`value + unit`”（例如 `50 mTorr`、`3 cm`、`500 V`）。
3. 统一输出：日志打印完整尺度表；输出文件中同时写 SI 与 code 值（可选）。

Acceptance：在同一算例中，用 SI 输入与用旧 code 输入得到一致的无量纲结果（误差在统计噪声内）。

Step 2：模块字典规范（Schema）与严格校验（优先）

Why：通用模块必须有清晰的配置契约；并且要为后续隐式/EM 扩展预留结构。
Deliverables：

1. 定义 `ExplicitPIC.dict` 规范（见下节建议结构），并提供：
   - 必填/可选字段、默认值、取值范围（例如形函数类型、子循环次数、MCC 开关等）。
   - “严格白名单”：未知字段报错（或至少警告）。
2. 实现 Schema/Validator：对字段类型（scalar/vector/block）、单位、范围进行校验。
3. 版本化：`schemaVersion` 字段，允许平滑升级。

Acceptance：对一组故意写错的 dict（缺字段/错单位/错类型）能给出清晰报错；对旧 PaperA dict 能通过兼容层迁移（打印 deprecations）。

Step 3：物种与常数数据库（为 MCC/隐式/EM 打底）

Why：MCC、迁移率、推导尺度都需要标准化的物种参数与常数。
Deliverables：

- `Species`：m, q、宏粒子权重、初始分布、温度/漂移、是否为电子/离子/中性。
- 常数表：$\epsilon_0, \mu_0, k_B, e$ 等。
- 统一 “气体/背景” 描述：气体种类、压力、温度、密度计算（理想气体）。

Acceptance：给定 p, T 自动算出 n_{gas} 并打印；与手算一致。

Step 4：MCC 框架骨架（优先）

Why：IEC 与放电类 benchmark 若无碰撞/反应，很难对标；框架需尽早定型以复用到隐式 PIC。
Deliverables：

1. CrossSectionDB：支持 “能量 eV $\rightarrow$ 截面” 曲线表 + 插值（线性/对数）。
2. Reaction DSL：用字典定义反应通道（elastic/excitation/ionization/CX），并绑定产物与能量分配模型。
3. ICollisionOperator：实现 null-collision/majorant 方案；对每个粒子给出 “是否发生事件” 与 “事件类型”。
4. 统计器：反应计数、反应率、能量损失；输出到 Diagnostics。

Acceptance：在均匀气体箱算例中，电子的平均碰撞频率与理论 $\nu = n\sigma v$ 一致（统计误差内）。

Step 5：最小反应集落地（e–neutral 弹性 + 电离）

Why：这是 “能跑放电/IEC 的最小集合”。
Deliverables：

- e–neutral elastic：散射角模型（各向同性/各向异性可选），能量更新。
- e–neutral ionization：生成二次电子 + 离子；能量分配（阈值 + 随机分配）。
- 粒子生成与注入统一走 SourceOperator（MCC 产生也算一种 source）。

Acceptance：Townsend-like 体电离算例（简化）：给定均匀电场，电离率随场强增大趋势正确；反应统计可复现。

Step 6：模块字典扩展到 “Sources + Boundaries + Circuits”（与 MCC 同期）

Why：MCC 产生的粒子、边界发射、外部束注入都需要统一配置结构。
Deliverables：

- Sources：`volumeInject`, `surfaceEmit`, `beamInject`, `mccIonization`（内部）。
- Boundaries：粒子-壁面模型(吸收/反射/再发射/SEE);场边界类型(Dirichlet/Neumann/Robin/Floating)。
- Circuits：最小 “CurrentBalance”（浮动电极）与 “VoltageDrive”（给定波形）。

Acceptance：1D sheath/diode：输出 $I(t)$ 与 IEDF；浮动电极电位收敛到合理值（电流平衡）。

Step 7：诊断体系升级（与回归指标固化）

Why：可分析性来自诊断，而不仅是云图。
Deliverables：

- 全局：总电荷、Gauss 残差（L1/L2/Linf）、能量（场能/动能/总能）与漂移率。
- 边界：按边界/物种的电流与能量通量；IEDF（分箱统计）。
- 反应：每通道反应率/能量损失；空间分布（可选）。
- 输出规范：统一文件命名与 units 元数据（便于后处理）。

Acceptance：plasma oscillation：频率与 ω_{pe} 一致；能量漂移在可控范围。

Step 8：AMR/粗细界面鲁棒性（paperA/paperB 风格问题前置防线）

Why：若后续要做隐式 PIC/更复杂算例，AMR 伪影（自力、界面噪声）会反复出现。
Deliverables：

- composite deposition / correction band 的落地（与你已讨论方向一致）。
- 在 coarse–fine 界面输出专项诊断：局部 Gauss 残差、局部能量与粒子统计。

Acceptance：自力检查算例：单粒子/对称分布在粗细界面附近无系统性加速（统计期望接近 0）。

Step 9：重启/检查点（工程闭环）

Why：长时间积分与参数扫必备；也是回归与调试的基础。
Deliverables：

- 序列化：粒子（x,v,w,label）、场（phi,E,rho）、RNG state、时间步信息。
- 重启一致性：重启后统计量连续，且与不重启版本一致（误差在噪声内）。

Acceptance：任意步写 checkpoint，重启继续跑，关键诊断曲线无跳变。

Step 10：IEC 专用能力补齐（面向 `iec_pic` 对标）

Why：IEC 需要更完整的电极/网格/注入/碰撞组合与特定诊断。
Deliverables：

- IEC 几何与边界模板（球/圆柱网格，栅极/阴极/阳极配置）。
- 离子注入与中性背景、CX 通道（若 IEC 工况需要）。
- IEC 输出：通过栅极电流、离子能谱、陷获率/损失率、空间电位结构。

Acceptance：复现 `iec_pic` 中至少 1 个代表性工况的主要曲线/统计指标。

Step 11：为隐式 PIC 预留“隐式接口替换点”

Why：隐式 PIC 不是在显式上堆补丁，而是替换“FieldSolver + Particle Advance + Coupling”。
Deliverables：

- 将“显式时间推进”写成可替换的 `IAdvanceScheme`（ExplicitLeapfrog / ImplicitECSIM 等）。
- 将“沉积/收集”抽象为可复用算子（隐式也要 matched deposit/gather）。
- 线性系统接口统一（隐式需要更多块矩阵/耦合项，但求解器接口不变）。

Acceptance：在不改 cases 的前提下，显式推进仍能跑；新接口对调用方透明。

Step 12：为 EM（Maxwell）扩展预留“场变量与求解器族”

Why：EM 需要 $\mathbf{E},\mathbf{B}$ 与 Maxwell 求解器，但 PICCore 的粒子/碰撞/源/边界很多可复用。
Deliverables：

- FieldState 抽象：ES（phi,E）与 EM（E,B, potentials）统一容器。
- IFieldSolver 支持“ESPoissonSolver”与“EMMaxwellSolver”并存。
- 边界模型拆分：场边界（电磁边界/导体）与粒子边界（吸收/SEE）组合。

Acceptance：最小 EM 测试（例如真空中波传播/能量守恒）可作为未来里程碑，但接口现在就要能承载。

4 字典规范建议（面向扩展：ES/Implicit/EM 共用骨架）

推荐将 PIC 模块字典分为 “通用块 + 具体算法块”：

```
PIC
{
  schemaVersion 1;
  units { mode SI; normalization problem; L0 3 cm; Phi0 500 V; n0 1e15 1/m^3;
     }

  species
  {
    e { charge -1 e; mass 1 me; weight 1e8; }
    Arp { charge +1 e; mass 40 amu; weight 1e8; }
  }

  background { gas Ar; pressure 50 mTorr; temperature 300 K; }

  numerics
  {
    shapeFunction CIC;     // NGP|CIC|TSC
    subcycling { e 10; Arp 1; }
    filters { rho none; E none; }
    diagnostics { energy true; gauss true; wallCurrent true; IEDF true; }
  }

  boundaries
  {
    cathode { field { type Dirichlet; phi 0 V; }
              particles { type Absorb; SEE { enabled true; model simple; yield
                  0.1; } } }
    anode   { field { type Dirichlet; phi 500 V; }
              particles { type Absorb; } }
    wall    { field { type Floating; circuit { type CurrentBalance; } }
              particles { type DiffuseReflect; T 300 K; } }
  }

  sources
  {
    beam1 { type BeamInject; species Arp; rate 1e18 1/s; energy 100 eV; }
  }

  collisions
  {
    enabled true;
    scheme nullCollision;
    reactions
```

```
    {
      e_Ar_elastic { type Elastic; projectile e; target Ar; xsTable "xs/e-Ar-
          elastic.dat"; }
      e_Ar_ion     { type Ionization; projectile e; target Ar; threshold 15.76
           eV;
                     products (e Arp); xsTable "xs/e-Ar-ion.dat"; }
    }
  }

  algorithm
  {
    family electrostatic;   // electrostatic|implicit_es|electromagnetic
    timeIntegration leapfrog; // leapfrog|boris|ecsim|... (future)
  }
}
```

5 推荐的最短落地顺序（浓缩版）

如果你希望 “尽快能做实际问题”，同时又不牺牲后续扩展，建议把工作压成四个短迭代：

1. **Iter-1（基础设施）：** Step 0–2（回归骨架 + Units + Dict Schema）。
2. **Iter-2（MCC 最小可用）：** Step 3–5（物种/常数 + MCC 框架 + 电离最小集）。
3. **Iter-3（可分析）：** Step 6–7（Sources/Boundaries/Circuits + 诊断固化）。
4. **Iter-4（可对标）：** Step 8–10（AMR 鲁棒 + 重启 + IEC 对标算例）。

完成 Iter-4 后，显式 PIC 基本具备 “通用可用 + 可对标 + 可扩展” 的工程形态；Step 11–12 则保证你在做隐式/EM 时不会推倒重来。

6 结语

你倾向把 “量纲、字典规范、MCC” 放前面是正确的：它们决定了整个系统的 “可表达性、可配置性与可扩展性”。按本路线推进，可以确保显式 PIC 不是一次性 demo，而是后续隐式 PIC 与电磁模块的可复用基座。